

CodeX · [Follow publication](#)

You're reading for free via [Hemanth Raju's](#) Friend Link. [Upgrade](#) to access the best of Medium.

 Member-only story

How to Build an AI Agent: From Idea to Real-World System

A practical guide to designing AI agents with the right model, memory, tools, orchestration, and evaluation systems.

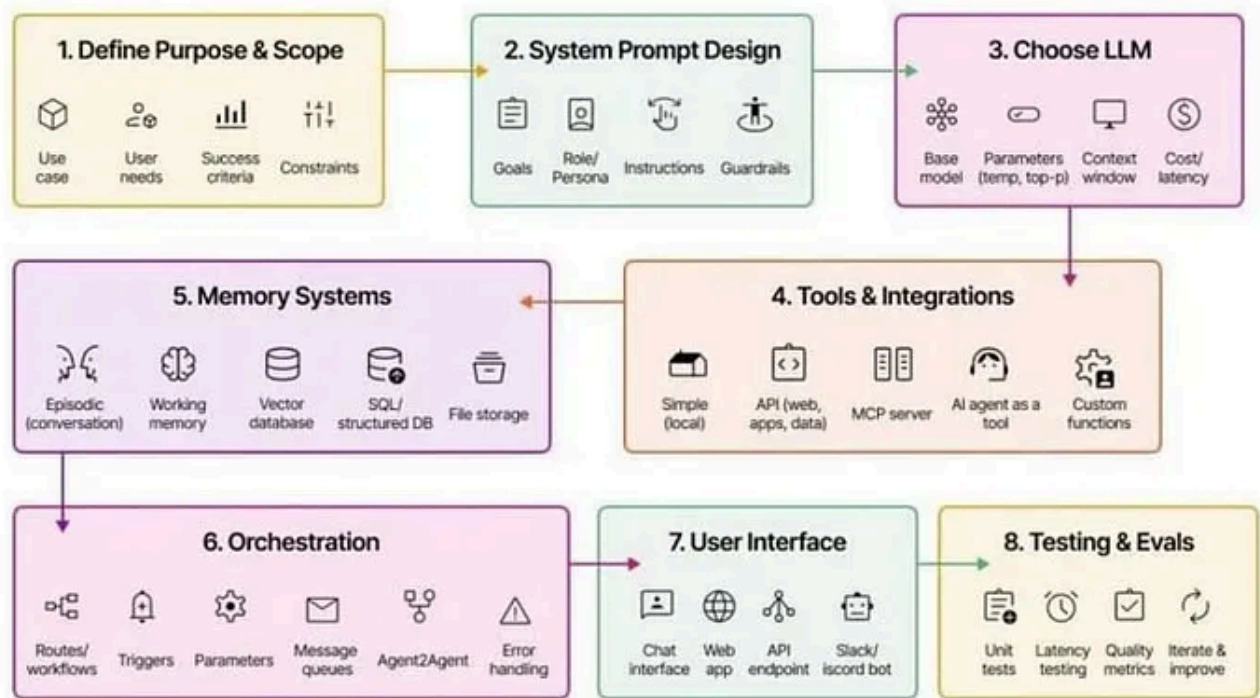
8 min read · Apr 28, 2026



Hemanth Raju

[Follow](#)[Listen](#)[Share](#)[... More](#)

How to Build an AI Agent



	Product/Platform	LLM	Deployment	Key Features	Key Features
Consumer AI Agents	ChatGPT (OpenAI)	GPT-5	Cloud	Custom GPTs, voice, vision, memory, DALL-E integration	General purpose assistant, creative work
	Claude (Anthropic)	Claude 4.5	Cloud	Projects, artifacts, analysis, 200K context	Research, writing, coding
	Perplexity	Multiple	Cloud	Search-first, citations, Pro search	Research assistant, fact-checking
Agentic Coding Tools	Cursor	Claude, GPT	Local + Cloud	Full IDE, composer mode, multi-file editing	Professional developers, complex projects
	Windsurf (Codeium)	Cascade	Local + Cloud	Flows, agentic editing, codebase awareness	Team development, large codebases
	Claude Code (Anthropic)	Claude 4.5	Local	Terminal-native, git integration, autonomous coding	CLI workflows, automation scripts
No-Code Builders	Lindy	GPT-5	Cloud	3000+ integrations, natural language workflows	Business automation, non-technical teams
	Relay.app	GPT-5	Cloud	Human-in-loop, Gmail/Slack native	Team workflows, approvals needed
	n8n	Multiple	Cloud / Both	400+ integrations, self-host, open source	Technical teams, data privacy needs
Development Frameworks	LangGraph	Any	Local/Cloud	Graph-based flows, state management, cycles	Complex workflows, production apps
	CrewAI	Any	Local/Cloud	Role-based, 40+ integrations, task delegation	Multi-agent teams, autonomous systems
	LlamaIndex	Any	Local/Cloud	RAG-first, data connectors, query engines	Knowledge-intensive apps, document Q&A

Everyone wants to build an AI agent right now.

Not just a chatbot. Not just a prompt wrapper.

A real AI agent — something that can understand goals, use tools, remember context, interact with users, and improve over time.

[Friend Link](#)

That's exactly what the image captures so well: building an agent is **not one decision**. It's a system design problem. An AI agent only becomes useful when several layers

work together — purpose, prompts, models, memory, orchestration, interfaces, and evaluation. 🧠 ⚙️

If you look closely, the image is really showing a blueprint. It says that an AI agent isn't built by picking an LLM and hoping for the best. It's built by carefully connecting **eight core parts** into a working whole.

Let's walk through that architecture the way a builder would.

It Starts With Purpose, Not Prompts

The first box in the image begins where many teams actually *should* begin, but often don't: **Define Purpose & Scope**.

This part includes:

- use case
- user needs
- success criteria
- constraints

That's more important than it may seem.

A lot of AI agent projects fail because they start with technology instead of clarity. Teams begin with “*Let's build an AI assistant*” without answering the harder questions:

- What problem is this agent solving?
- Who is it helping?
- What does success actually look like?
- What boundaries does it need to respect?

An agent designed for customer support is fundamentally different from an agent built for coding, scheduling, or internal knowledge retrieval. Once the purpose is clear, every other design choice becomes easier. 🎯

In other words, the first step in building an AI agent is not prompting — it's **product thinking**.



The System Prompt Is the Agent's Operating Manual

The second stage in the image is **System Prompt Design**, and this is where the agent begins to take shape.

The image highlights:

- goals
- role/persona
- instructions
- guardrails

This matters because the system prompt is what turns a generic language model into a specific kind of agent. It defines how the agent should behave, what it should prioritize, what tone it should use, and where it should stop.

A good system prompt doesn't just tell the model what to do. It gives the model a stable identity inside the workflow. It tells the model:

- what role it plays
- what kind of output is expected
- how careful it should be
- what actions are allowed or forbidden

Without this layer, agents feel inconsistent. With it, they become reliable and focused. 🕒

The strongest AI agents are rarely powered by “one clever prompt.” They are guided by carefully designed operating instructions.



Choosing the Right LLM Is a Trade-Off, Not a Trophy Contest

The third step in the image is **Choose LLM**, and the subtext here is important. The image points to:

- base model
- parameters (temperature, top-p)

- context window
- cost / latency

This is a smart reminder that choosing an LLM is not about picking the “most powerful” one in the abstract. It’s about choosing the right intelligence for the job.

Some agents need:

- deep reasoning
- long context handling
- strong coding ability
- low latency
- lower inference cost

Others need more lightweight models because the tasks are repetitive, predictable, or high-volume.

The image even hints at different categories of platforms later on: consumer assistants, agentic coding tools, no-code builders, and developer frameworks. That’s a good signal that model choice is tied to the kind of agent you’re building. 🧩

A research assistant may need a model with strong reasoning and document understanding.

A workflow automation bot may care more about speed and integrations.

A coding agent may need a model optimized for code editing and repo awareness.

The best builders don’t ask, “*What’s the smartest model?*”

They ask, “*What model makes this agent practical?*”

🔑 **Tools and Integrations Turn Intelligence Into Action**

This is where things get real.

The image’s fourth block is **Tools & Integrations**, and it includes:

- simple local tools
- APIs (web, apps, data)

- MCP server
- AI agent as a tool
- custom functions

This is the moment when an AI agent stops being just a responder and starts becoming an **actor**.

Without tools, an agent can only generate text.

With tools, an agent can:

- retrieve live information
- call APIs
- update systems
- execute workflows
- interact with external data
- delegate tasks to other agents

This is one of the most important ideas in the whole image. AI agents become useful not because they “sound intelligent,” but because they can **do something in the world**. 🌍

The mention of an **MCP server** is especially notable because it suggests a structured way for agents to access tools and context. That means tools aren’t just random additions — they’re part of a governed, repeatable system.

And the idea of “**AI agent as a tool**” hints at something even more advanced: one agent can call another specialized agent when needed. That’s how single-agent systems evolve into collaborative architectures. 🧡

🧠 **Memory Is What Makes an Agent Feel Persistent**

The fifth block in the image is **Memory Systems**, and this is where many people start to understand why AI agents feel different from simple chatbots.

The memory types shown are:

- episodic memory (conversation)

- working memory
- vector database
- SQL / structured DB
- file storage

That's a sophisticated picture of memory, because not all memory is the same.

Episodic memory

This is the conversational history — the short-term record of what has been said and done.

Working memory

This is the temporary scratchpad the agent uses while solving the current task.

Vector database

This supports semantic retrieval, allowing the agent to recall documents, notes, or information by meaning rather than exact keywords.

SQL / structured database

This is where structured facts, records, and operational state can live.

File storage

This supports larger artifacts — documents, spreadsheets, outputs, reports, or long-form working material.

Together, these memory systems make an agent feel persistent rather than forgetful. They allow it to remember what matters, retrieve what's relevant, and continue work over time. 🕒

An AI agent without memory is a session.

An AI agent with memory becomes a system.

Orchestration Is the Hidden Engine

The sixth section of the image is **Orchestration**, and this may be the most underrated part of building an AI agent.

The image includes:

- routes / workflows

- triggers
- parameters
- message queues
- agent2agent
- error handling

This is the machinery that keeps an agent operational.

Most useful AI agents are not one-step systems. They need logic such as:

- if this happens, trigger that
- if the response fails, retry
- if the request is complex, route to another path
- if another agent is needed, hand off the task
- if external latency is high, manage queues
- if the tool call breaks, recover gracefully

This is why orchestration matters so much. It is the difference between a clever demo and a production system. 

The mention of **Agent2Agent** is particularly important. It suggests that orchestration is not only about connecting functions — it's also about coordinating multiple intelligent components. That's where agent systems begin to resemble distributed software.

The Interface Determines Whether People Actually Use the Agent

The seventh layer is **User Interface**, and the image shows several ways users may interact with an agent:

- chat interface
- web app
- API endpoint
- Slack / Discord bot

This is a reminder that an agent is not just intelligence — it is also an experience.

A great internal knowledge agent might live inside Slack.

A customer-facing support assistant might need a web interface.

A backend automation agent might expose only an API.

A personal copilot might live in a chat interface.

The UI affects how users trust, understand, and control the agent. If the experience is awkward, even a smart agent feels unusable. If the interface is intuitive, the same agent feels powerful and accessible. 💡

In practice, this means agent design is not only about backend architecture. It's also about meeting users where they already work.

✅ **Testing and Evals Are What Make Agents Reliable**

The final block in the architecture is **Testing & Evals**, and the image includes:

- unit tests
- latency testing
- quality metrics
- iterate & improve

This is one of the most mature parts of the image because it reinforces a truth many teams learn the hard way:

AI agents cannot be trusted in production without evaluation.

An agent may look impressive in a demo and still fail badly in real usage. Testing helps answer critical questions:

- Does the agent follow instructions consistently?
- Are tool calls correct?
- Is the latency acceptable?
- Does output quality hold up across cases?
- Are improvements actually making the system better?

This section makes it clear that agent building is not a one-and-done activity. It is iterative. You build, test, measure, refine, and repeat. 

The best AI teams don't just launch agents.
They evolve them.

The Tool Landscape in the Image Tells a Bigger Story

At the bottom, the image includes a table of platforms and frameworks across several categories. Even without turning that section into a directory, the signal is powerful.

It shows that the AI agent ecosystem is now broad enough to support different types of builders:

Consumer AI assistants

Tools like ChatGPT, Claude, and Perplexity appear as general-purpose systems for assistance, research, reasoning, and search. 

Agentic coding tools

Platforms like Cursor, Windsurf (Codeium), and Claude Code reflect the rise of agents built specifically for software development — editing codebases, running terminal workflows, and supporting complex engineering tasks. 

No-code and workflow builders

Entries like Lindy, Relay.app, and n8n show that agents are no longer just for engineers. Teams can now build workflow agents and automation systems without heavy coding. 

Developer frameworks

Frameworks like LangGraph, CrewAI, and LlamaIndex suggest a more composable future — where teams design graph-based workflows, role-based delegation, RAG-first systems, and knowledge-intensive agent architectures. 

That bottom section tells a bigger story: AI agents are no longer one category of product. They are becoming an entire ecosystem of tools for different types of work.

What the Image Really Teaches

What makes this visual powerful is that it quietly corrects a common misconception.

People often think building an AI agent means:

pick a model + write a prompt

But the image shows something much more important:

Building an AI agent means designing a **complete system**.

You need:

- a defined purpose
- a structured prompt
- the right model
- tools and integrations
- memory
- orchestration
- interface design
- testing and iteration

That's why AI agent development feels closer to product engineering than to simple prompt engineering. It sits at the intersection of software, UX, workflows, reasoning, and operations. 

Final Thoughts

The biggest takeaway from this image is simple:

An AI agent is not a feature. It is an architecture.

The best agents are not built by accident. They are designed deliberately — starting from purpose, moving through prompt and model choices, gaining usefulness through tools and memory, becoming dependable through orchestration, and earning trust through testing.  

That's why the future of AI won't just be about smarter models.
It will be about better systems built around those models.

And this image captures that shift beautifully.

Thanks for reading ! I hope you enjoyed  reading this article.

You can subscribe [here](#) for regular articles .

Let's connect via [GitHub](#) and [Linkedin](#).

Keep smiling 😁!

Have a nice day!

Programming

Technology

Software Development

Artificial Intelligence

Data Science



Follow

Published in CodeX

34K followers · Last published 6 hours ago

Everything connected with Tech & Code. Follow to join our 1M+ monthly readers



Follow



Written by Hemanth Raju

1.6K followers · 6.8K following

Software Engineer / Tech Blogger / 🌐 LinkedIn: <https://www.linkedin.com/in/hemanth-raju-koneti/>
https://www.youtube.com/@khr_codes, buymeacoffee.com/khrworks

Open in app ↗



Medium

